In many cases you can do some interesting calculations on data from a single system to get new insight into recidivism. An example using only SCM data is the Comment To Commit Ratio. If a team is using a workflow that includes code reviews, sometimes developers, usually the leads, will end up spending most of their time reviewing other people's code rather than writing any code themselves. We call this PR Paralysis. This is usually a bad sign that there isn't enough senior technical leadership on the team or pull requests aren't being distributed across enough of your team. You can calculate Comment To Commit Ratio with the following formula:

```
Comment To Commit Ratio = r / (m + c)
```

- `m` = merged pull requests
- `c` = commits
- `r` = reviews

In chapter 8 we'll look at the elements of good software, and one of the measures we'll talk about is Mean Time To Repair (MTTR). This is another important metric that can be calculated simply with data from your APM if you just want it at a high level. The most simplistic view of MTTR would be

```
MTTR = f - s
```

- `s` = start date time when an anomaly is identified
- `f` = date time when the anomaly is fixed

Later in this chapter we'll be looking at estimate health, or how accurate your team's estimations are. The algorithm for estimate health is outlined in listing 7.1. In a few words it compares the amount of time a task took to complete against the estimated effort, and gives a rating of 0 when estimates line up, a rating of greater than 0 when tasks are taking longer than estimated (underestimating), and a rating of less than 0 when tasks take less time than estimated (overestimating).

A rather complex metric that we'll dive into in our case study is release health. This is a combination of PTS, SCM, and release data to find out how